

Lecture 1: Matrix Decompositions

Jinwoo Shin

Kim Jaechul Graduate School of AI, KAIST

- (1) Determinant and Trace
- (2) Eigenvalues and Eigenvectors
- (3) Cholesky Decomposition
- (4) Eigendecomposition and Diagonalization
- (5) Singular Value Decomposition

- How to summarize matrices: determinants and eigenvalues
- How matrices can be decomposed: Cholesky decomposition, diagonalization, singular value decomposition

- (1) Determinant and Trace
- (2) Eigenvalues and Eigenvectors
- (3) Cholesky Decomposition
- (4) Eigendecomposition and Diagonalization
- (5) Singular Value Decomposition

Determinant: Motivation (1)

- For $\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}$, $\mathbf{A}^{-1} = \frac{1}{a_{11}a_{22} - a_{12}a_{21}} \begin{pmatrix} a_{22} & -a_{12} \\ -a_{21} & a_{11} \end{pmatrix}$.
- $\mathbf{A}$ is invertible iff $a_{11}a_{22} - a_{12}a_{21} \neq 0$
- Let's define $\det(\mathbf{A}) = a_{11}a_{22} - a_{12}a_{21}$.
- Notation: $\det(\mathbf{A})$ or $|\text{whole matrix}|$
- What about 3×3 matrix? By doing some algebra (e.g., Gaussian elimination),

$$\begin{vmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{vmatrix} = a_{11}a_{22}a_{33} + a_{21}a_{32}a_{13} + a_{31}a_{12}a_{23} \\ - a_{31}a_{22}a_{13} - a_{11}a_{32}a_{23} - a_{21}a_{12}a_{33}$$

Determinant: Motivation (2)

- Try to find some pattern ...

$$\begin{aligned}
 & a_{11}a_{22}a_{33} + a_{21}a_{32}a_{13} + a_{31}a_{12}a_{23} \\
 & - a_{31}a_{22}a_{13} - a_{11}a_{32}a_{23} - a_{21}a_{12}a_{33} = \\
 & a_{11}(-1)^{1+1} \det(\mathbf{A}_{1,1}) + a_{12}(-1)^{1+2} \det(\mathbf{A}_{1,2}) \\
 & + a_{13}(-1)^{1+3} \det(\mathbf{A}_{1,3})
 \end{aligned}$$

- $\mathbf{A}_{k,j}$ is the submatrix of $\mathbf{A}$ that we obtain when deleting row k and column j .

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \text{ gives the term } a_{11} \begin{vmatrix} a_{22} & a_{23} \\ a_{32} & a_{33} \end{vmatrix}$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \text{ gives the term } a_{12} \left(- \begin{vmatrix} a_{21} & a_{23} \\ a_{31} & a_{33} \end{vmatrix} \right)$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \text{ gives the term } a_{13} \begin{vmatrix} a_{21} & a_{22} \\ a_{31} & a_{32} \end{vmatrix}$$

source: www.cliffsnotes.com

- This is called [Laplace expansion](#).
- Now, we can generalize this and provide the formal definition of determinant.

Determinant

For a matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, for all $j = 1, \dots, n$,

1. Expansion along column j : $\det(\mathbf{A}) = \sum_{k=1}^n (-1)^{k+j} a_{kj} \det(\mathbf{A}_{k,j})$
2. Expansion along row j : $\det(\mathbf{A}) = \sum_{k=1}^n (-1)^{k+j} a_{jk} \det(\mathbf{A}_{j,k})$

- All expansion are equal, so no problem with the definition.
- **Theorem.** $\det(\mathbf{A}) \neq 0 \iff \text{rk}(\mathbf{A}) = n \iff \mathbf{A}$ is invertible.

Determinant: Properties

- (1) $\det(\mathbf{AB}) = \det(\mathbf{A}) \det(\mathbf{B})$
- (2) $\det(\mathbf{A}) = \det(\mathbf{A}^T)$
- (3) For a regular $\mathbf{A}$, $\det(\mathbf{A}^{-1}) = 1 / \det(\mathbf{A})$
- (4) For two similar matrices $\mathbf{A}, \mathbf{A}'$ (i.e., $\mathbf{A}' = \mathbf{S}^{-1} \mathbf{A} \mathbf{S}$ for some $\mathbf{S}$), $\det(\mathbf{A}) = \det(\mathbf{A}')$
- (5) For a triangular matrix¹ $\mathbf{T}$, $\det(\mathbf{T}) = \prod_{i=1}^n T_{ii}$
- (6) Adding a multiple of a column/row to another one does not change $\det(\mathbf{A})$
- (7) Multiplication of a column/row with λ scales $\det(\mathbf{A})$: $\det(\lambda \mathbf{A}) = \lambda^n \det(\mathbf{A})$
- (8) Swapping two rows/columns changes the sign of $\det(\mathbf{A})$
 - Using (5)-(8), Gaussian elimination (reaching a triangular matrix) enables to compute the determinant.

¹This includes diagonal matrices.

- **Definition.** The trace of a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ is defined as

$$\text{tr}(\mathbf{A}) := \sum_{i=1}^n a_{ii}$$

- $\text{tr}(\mathbf{A} + \mathbf{B}) = \text{tr}(\mathbf{A}) + \text{tr}(\mathbf{B})$
- $\text{tr}(\alpha \mathbf{A}) = \alpha \text{tr}(\mathbf{A})$
- $\text{tr}(\mathbf{I}_n) = n$

- (1) Determinant and Trace
- (2) Eigenvalues and Eigenvectors
- (3) Cholesky Decomposition
- (4) Eigendecomposition and Diagonalization
- (5) Singular Value Decomposition

- **Definition.** Consider a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$. Then, $\lambda \in \mathbb{R}$ is an eigenvalue of $\mathbf{A}$ and $\mathbf{x} \in \mathbb{R}^n \setminus \{0\}$ is the corresponding eigenvector of $\mathbf{A}$ if

$$\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$$

- Equivalent statements
 - λ is an eigenvalue.
 - $(\mathbf{A} - \lambda\mathbf{I}_n)\mathbf{x} = 0$ can be solved non-trivially, i.e., $\mathbf{x} \neq 0$.
 - $\det(\mathbf{A} - \lambda\mathbf{I}_n) = 0$

Example

- For $\mathbf{A} = \begin{pmatrix} 4 & 2 \\ 1 & 3 \end{pmatrix}$, $p_{\mathbf{A}}(\lambda) = \begin{vmatrix} 4 - \lambda & 2 \\ 1 & 3 - \lambda \end{vmatrix} = (4 - \lambda)(3 - \lambda) - 2 \cdot 1 = \lambda^2 - 7\lambda + 10$
- Eigenvalues $\lambda = 2$ or $\lambda = 5$.
- Eigenvector E_5 for $\lambda = 5$
$$\begin{pmatrix} 4 - \lambda & 2 \\ 1 & 3 - \lambda \end{pmatrix} \mathbf{x} = 0 \implies \begin{pmatrix} -1 & 2 \\ 1 & -2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = 0 \implies E_5 = \text{span}\left[\begin{pmatrix} 2 \\ 1 \end{pmatrix}\right]$$
- Eigenvector E_2 for $\lambda = 2$. Similarly, we get $E_2 = \text{span}\left[\begin{pmatrix} 1 \\ -1 \end{pmatrix}\right]$
- **Message.** Eigenvectors are not unique.

- For $\mathbf{A} \in \mathbb{R}^{n \times n}$, n distinct eigenvalues $\implies$ eigenvectors are linearly independent, which form a basis of $\mathbb{R}^n$.
 - Converse is not true.
 - Example of n linearly independent eigenvectors for less than n eigenvalues???
- **Determinant**. For (possibly repeated) eigenvalues λ_i of $\mathbf{A} \in \mathbb{R}^{n \times n}$,

$$\det(\mathbf{A}) = \prod_{i=1}^n \lambda_i$$

- **Trace**. For (possibly repeated) eigenvalues λ_i of $\mathbf{A} \in \mathbb{R}^{n \times n}$,

$$\text{tr}(\mathbf{A}) = \sum_{i=1}^n \lambda_i$$

- (1) Determinant and Trace
- (2) Eigenvalues and Eigenvectors
- (3) Cholesky Decomposition
- (4) Eigendecomposition and Diagonalization
- (5) Singular Value Decomposition

Cholesky Decomposition

- A real number: decomposition of two identical numbers, e.g., $9 = 3 \times 3$
- **Theorem.** For a symmetric, positive definite matrix $\mathbf{A}$, $\mathbf{A} = \mathbf{L}\mathbf{L}^T$, where
 - $\mathbf{L}$ is a lower-triangular matrix with positive diagonals
 - Such a $\mathbf{L}$ is unique, called **Cholesky factor** of $\mathbf{A}$.
- Applications
 - (a) factorization of covariance matrix of a multivariate Gaussian variable
 - (b) linear transformation of random variables
 - (c) fast determinant computation: $\det(\mathbf{A}) = \det(\mathbf{L}) \det(\mathbf{L}^T) = \det(\mathbf{L})^2$, where $\det(\mathbf{L}) = \prod_i l_{ii}$. Thus, $\det(\mathbf{A}) = \prod_i l_{ii}^2$.

- (1) Determinant and Trace
- (2) Eigenvalues and Eigenvectors
- (3) Cholesky Decomposition
- (4) Eigendecomposition and Diagonalization
- (5) Singular Value Decomposition

Diagonal Matrix and Diagonalization

- **Diagonal matrix.** zero on all off-diagonal elements, $\mathbf{D} = \begin{pmatrix} d_1 & \cdots & 0 \\ \vdots & & \vdots \\ 0 & \cdots & d_n \end{pmatrix}$

$$\mathbf{D}^k = \begin{pmatrix} d_1^k & \cdots & 0 \\ \vdots & & \vdots \\ 0 & \cdots & d_n^k \end{pmatrix}, \quad \mathbf{D}^{-1} = \begin{pmatrix} 1/d_1 & \cdots & 0 \\ \vdots & & \vdots \\ 0 & \cdots & 1/d_n \end{pmatrix}, \quad \det(\mathbf{D}) = d_1 d_2 \cdots d_n$$

- **Definition.** $\mathbf{A} \in \mathbb{R}^{n \times n}$ is **diagonalizable** if it is similar to a diagonal matrix $\mathbf{D}$, i.e., $\exists$ an **invertible** $\mathbf{P} \in \mathbb{R}^{n \times n}$, such that $\mathbf{D} = \mathbf{P}^{-1} \mathbf{A} \mathbf{P}$.
- **Definition.** $\mathbf{A} \in \mathbb{R}^{n \times n}$ is **orthogonally diagonalizable** if it is similar to a diagonal matrix $\mathbf{D}$, i.e., $\exists$ an **orthogonal** $\mathbf{P} \in \mathbb{R}^{n \times n}$, such that $\mathbf{D} = \mathbf{P}^{-1} \mathbf{A} \mathbf{P} = \mathbf{P}^T \mathbf{A} \mathbf{P}$.

- $\mathbf{A}^k = \mathbf{P}\mathbf{D}^k\mathbf{P}^{-1}$
- $\det(\mathbf{A}) = \det(\mathbf{P}) \det(\mathbf{D}) \det(\mathbf{P}^{-1}) = \det(\mathbf{D}) = \prod_i d_{ii}$
- Many other things ...
- **Question.** Under what condition is $\mathbf{A}$ diagonalizable (or orthogonally diagonalizable) and how can we find $\mathbf{P}$ (thus $\mathbf{D}$)?

Orthogonally Diagonalizable and Symmetric Matrix

Theorem. $\mathbf{A} \in \mathbb{R}^{n \times n}$ is orthogonally diagonalizable $\iff \mathbf{A}$ is symmetric.

- **Question.** How to find $\mathbf{P}$ (thus $\mathbf{D}$)?
- **Spectral Theorem.** If $\mathbf{A} \in \mathbb{R}^{n \times n}$ is symmetric,
 - (a) the eigenvalues are all real
 - (b) the eigenvectors to different eigenvalues are perpendicular.
 - (c) there exists an orthogonal eigenbasis
- The above implies the columns of $\mathbf{P}$ are the n eigenvectors of $\mathbf{A}$ because $\mathbf{AP} = \mathbf{PD}$ and $\mathbf{P}^T = \mathbf{P}^{-1}$ ($\mathbf{P}$ is an orthogonal matrix).

- (1) Determinant and Trace
- (2) Eigenvalues and Eigenvectors
- (3) Cholesky Decomposition
- (4) Eigendecomposition and Diagonalization
- (5) **Singular Value Decomposition**

- Eigendecomposition (also called EVD: EigenValue Decomposition): (Orthogonal) Diagonalization for symmetric matrices $\mathbf{A} \in \mathbb{R}^{n \times n}$.
- Extensions: Singular Value Decomposition (SVD)
 1. First extension: diagonalization for non-symmetric, but still square matrices $\mathbf{A} \in \mathbb{R}^{n \times n}$
 2. Second extension: diagonalization for non-symmetric, and non-square matrices $\mathbf{A} \in \mathbb{R}^{m \times n}$
- **Background.** For $\mathbf{A} \in \mathbb{R}^{m \times n}$, a matrix $\mathbf{S} := \mathbf{A}^T \mathbf{A} \in \mathbb{R}^{n \times n}$ is always symmetric, positive semidefinite.
 - Symmetric, because $\mathbf{S}^T = (\mathbf{A}^T \mathbf{A})^T = \mathbf{A}^T \mathbf{A} = \mathbf{S}$.
 - Positive semidefinite, because $\mathbf{x}^T \mathbf{S} \mathbf{x} = \mathbf{x}^T \mathbf{A}^T \mathbf{A} \mathbf{x} = (\mathbf{A} \mathbf{x})^T (\mathbf{A} \mathbf{x}) \geq 0$.

Singular Value Decomposition

- **Theorem.** $\mathbf{A} \in \mathbb{R}^{m \times n}$ with rank $r \in [0, \min(m, n)]$. The SVD of $\mathbf{A}$ is a decomposition of the form

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T, \quad \left| \quad \begin{array}{c} n \\ \boxed{\mathbf{A}} \\ m \end{array} = \begin{array}{c} m \\ \boxed{\mathbf{U}} \\ m \end{array} \begin{array}{c} m \\ \boxed{\mathbf{\Sigma}} \\ m \end{array} \begin{array}{c} n \\ \boxed{\mathbf{V}^T} \\ n \end{array} \right.$$

with an orthogonal matrix $\mathbf{U} = (\mathbf{u}_1 \cdots \mathbf{u}_m) \in \mathbb{R}^{m \times m}$ and an orthogonal matrix $\mathbf{V} = (\mathbf{v}_1 \cdots \mathbf{v}_n) \in \mathbb{R}^{n \times n}$. Moreover, $\mathbf{\Sigma}$ is an $m \times n$ matrix with $\Sigma_{ii} = \sigma_i \geq 0$ and $\Sigma_{ij} = 0$, $i \neq j$, which is uniquely determined for $\mathbf{A}$.

- Note
 - The diagonal entries σ_i , $i = 1, \dots, r$ are called **singular values**.
 - $\mathbf{u}_i$ and $\mathbf{v}_j$ are called **left** and **right singular vectors**, respectively.

SVD: How It Works (for $\mathbf{A} \in \mathbb{R}^{n \times n}$)

- $\mathbf{A} \in \mathbb{R}^{n \times n}$ with rank $r \leq n$. Then, $\mathbf{A}^T \mathbf{A}$ is symmetric.
- Orthogonal diagonalization of $\mathbf{A}^T \mathbf{A}$:

$$\mathbf{A}^T \mathbf{A} = \mathbf{V} \mathbf{D} \mathbf{V}^T.$$

- $\mathbf{D} = \begin{pmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_n \end{pmatrix}$ and an orthogonal matrix $\mathbf{V} = (\mathbf{v}_1 \cdots \mathbf{v}_n)$, where $\lambda_1 \geq \cdots \geq \lambda_r \geq \lambda_{r+1} = \cdots = \lambda_n = 0$ are the eigenvalues of $\mathbf{A}^T \mathbf{A}$ and $\{\mathbf{v}_i\}$ are orthonormal.
- All λ_i are positive
 $\forall \mathbf{x} \in \mathbb{R}^n, \|\mathbf{A}\mathbf{x}\|^2 = \mathbf{A}\mathbf{x}^T \mathbf{A}\mathbf{x} = \mathbf{x}^T \mathbf{A}^T \mathbf{A}\mathbf{x} = \lambda_i \|\mathbf{x}\|^2$

- $\text{rk}(\mathbf{A}) = \text{rk}(\mathbf{A}^T \mathbf{A}) = \text{rk}(\mathbf{D}) = r$
- Choose $\mathbf{U}' = (\mathbf{u}_1 \cdots \mathbf{u}_r)$, where

$$\mathbf{u}_i = \frac{\mathbf{A}\mathbf{v}_i}{\sqrt{\lambda_i}}, \quad 1 \leq i \leq r.$$

- We can construct $\{\mathbf{u}_i\}$, $i = r+1, \dots, n$, so that $\mathbf{U} = (\mathbf{u}_1 \cdots \mathbf{u}_n)$ is an orthonormal basis of $\mathbb{R}^n$.
- Define $\Sigma = \begin{pmatrix} \sqrt{\lambda_1} & & \\ & \ddots & \\ & & \sqrt{\lambda_n} \end{pmatrix}$
- Then, we can check that $\mathbf{U}\Sigma = \mathbf{A}\mathbf{V}$.
- Similar arguments for a general $\mathbf{A} \in \mathbb{R}^{m \times n}$

EVD ($\mathbf{A} = \mathbf{P}\mathbf{D}\mathbf{P}^{-1}$) vs. SVD ($\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$)

- SVD: **always** exists, EVD: **square** matrix and exists if we can find **a basis of eigenvectors** (such as symmetric matrices)
- $\mathbf{P}$ in EVD is **not necessarily orthogonal** (only true for symmetric $\mathbf{A}$), but $\mathbf{U}$ and $\mathbf{V}$ are **orthogonal** (so representing rotations)
- Both EVD and SVD: (i) basis change in the domain, (ii) independent scaling of each new basis vector and mapping from domain to codomain, (iii) basis change in the codomain. The difference: for SVD, **different vector spaces** of domain and codomain.
- SVD and EVD are closely related through their projections
 - The left-singular (resp. right-singular) vectors of $\mathbf{A}$ are eigenvectors of $\mathbf{A}\mathbf{A}^T$ (resp. $\mathbf{A}^T\mathbf{A}$)
 - The singular values of $\mathbf{A}$ are the square roots of eigenvalues of $\mathbf{A}\mathbf{A}^T$ and $\mathbf{A}^T\mathbf{A}$
 - When $\mathbf{A}$ is symmetric, EVD = SVD (from spectral theorem)

Questions?

Lecture 2: Convex Optimization

Jinwoo Shin

Kim Jaechul Graduate School of AI, KAIST

- (1) Optimization Using Gradient Descent
- (2) Constrained Optimization and Lagrange Multipliers
- (3) Convex Sets and Functions
- (4) Convex Optimization

- Training machine learning models = finding a good set of parameters
- A good set of parameters = Solution (or close to solution) to some optimization problem
- Directions: Unconstrained optimization, Constrained optimization, Convex optimization
- High-school math: A necessary condition for the optimal point: $f'(x) = 0$ (stationary point)
 - Gradient will play an important role

- (1) Optimization Using Gradient Descent
- (2) Constrained Optimization and Lagrange Multipliers
- (3) Convex Sets and Functions
- (4) Convex Optimization

Unconstrained Optimization and Gradient Algorithms

- Goal

$$\min f(\mathbf{x}), \quad f(\mathbf{x}) : \mathbb{R}^n \mapsto \mathbb{R}, \quad f \in C^1$$

- Gradient-type algorithms

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \gamma_k \mathbf{d}_k, \quad k = 0, 1, 2, \dots$$

- **Lemma.** Any direction $\mathbf{d} \in \mathbb{R}^{n \times 1}$ that satisfies $\nabla f(\mathbf{x}) \cdot \mathbf{d} < 0$ is a descent direction of f at $\mathbf{x}$. That is, if we let $\mathbf{x}_\alpha = \mathbf{x} + \alpha \mathbf{d}$, $\exists \bar{\alpha} > 0$, such that for all $\alpha \in (0, \bar{\alpha}]$, $f(\mathbf{x}_\alpha) < f(\mathbf{x})$.
- Steepest gradient descent¹. $\mathbf{d}_k = -\nabla f(\mathbf{x}_k)^\top$.
- Finding a local optimum $f(\mathbf{x}_*)$, if the step-size γ_k is suitably chosen.

¹In some cases, just gradient descent often means this steepest gradient descent.

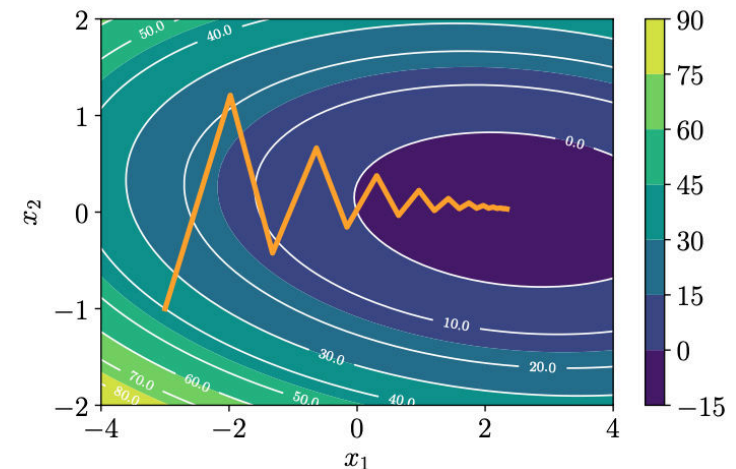
Example

- A quadratic function $f : \mathbb{R}^2 \mapsto \mathbb{R}$.

$$f \left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \right) = \frac{1}{2} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}^T \begin{pmatrix} 2 & 1 \\ 1 & 20 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} - \begin{pmatrix} 5 \\ 3 \end{pmatrix}^T \begin{pmatrix} x_1 \\ x_2 \end{pmatrix},$$

whose gradient is $\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}^T \begin{pmatrix} 2 & 1 \\ 1 & 20 \end{pmatrix} - \begin{pmatrix} 5 \\ 3 \end{pmatrix}^T$

- $\mathbf{x}_0 = (-3 - 1)^T$
- constant step size $\alpha = 0.085$
- Zigzag pattern



- Goal: $\min L(\theta)$ for n training data
- Based on the **amount of training data** used for **each** iteration
 - Batch gradient descent (the entire n)
 - Mini-batch gradient descent ($k < n$ data)
 - Stochastic gradient descent ($k < n$ data with unbiased gradient estimation)
- Based on the adaptive method of update
 - Momentum, NAG, Adagrad, RMSprop, Adam, etc
- <https://ruder.io/optimizing-gradient-descent/>

Stochastic Gradient Descent (SGD)

- Assume $L(\boldsymbol{\theta}) = \sum_{i=1}^n L_n(\boldsymbol{\theta})$ (which happens in many cases in machine learning)
- Gradient update

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \gamma_k \nabla L(\boldsymbol{\theta}_k)^\top = \boldsymbol{\theta}_k - \gamma_k \sum_{n=1}^N \nabla L_n(\boldsymbol{\theta}_k)^\top$$

- Batch gradient: $\sum_{n=1}^N \nabla L_n(\boldsymbol{\theta}_k)^\top$
- Mini-batch gradient: $\sum_{n \in \mathcal{K}} \nabla L_n(\boldsymbol{\theta}_k)^\top$ for a suitable choice of $\mathcal{K}$, $|\mathcal{K}| < n$
- Stochastic gradient: Randomly choose the subset $\mathcal{K}$ of mini-batch gradient such that

$$\sum_{n=1}^N \nabla L_n(\boldsymbol{\theta}_k)^\top = E \left[\sum_{n \in \mathcal{K}} \nabla L_n(\boldsymbol{\theta}_k)^\top \right]$$

i.e., noisy approximation to the real gradient.

Adaptivity for Better Convergence: Momentum

- Step size.
 - Too small: slow update, Too big: overshoot, zig-zag, often fail to converge
- Adaptive update: smooth out the erratic behavior and dampens oscillations
- Gradient descent with **momentum**

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \gamma_i \nabla f(\mathbf{x}_k)^\top + \alpha \Delta \mathbf{x}_k, \quad \alpha \in [0, 1]$$

$$\Delta \mathbf{x}_k = \mathbf{x}_k - \mathbf{x}_{k-1}$$

- Memory term: $\alpha \Delta \mathbf{x}_k$, where α is the degree of how much we remember the past
- Next update = a linear combination of current and previous updates

- (1) Optimization Using Gradient Descent
- (2) **Constrained Optimization and Lagrange Multipliers**
- (3) Convex Sets and Functions
- (4) Convex Optimization

Standard Constrained Optimization Problem

- An optimization problem in standard form:

minimize $f(\mathbf{x})$

subject to $g_i(\mathbf{x}) \leq 0, \quad i = 1, 2, \dots, m$ (*Inequality constraints*)

$h_j(\mathbf{x}) = 0, \quad j = 1, 2, \dots, p$ (*Equality constraints*)

- Variables: $\mathbf{x} \in \mathbb{R}^n$. Assume nonempty feasible set
- Optimal value: p^* . Optimizer: $\mathbf{x}^*$

Problem Solving via Lagrange Multipliers

- Duality Mentality
 - Bound or solve an optimization problem via a different optimization problem!
 - We'll develop the basic Lagrange duality theory for a general optimization problem, then specialize for convex optimization

- Idea: augment the objective with a weighted sum of constraints

- Lagrangian:

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\nu}) = f(\mathbf{x}) + \sum_{i=1}^m \lambda_i g_i(\mathbf{x}) + \sum_{i=1}^p \nu_i h_i(\mathbf{x})$$

- Lagrange multipliers (dual variables): $\boldsymbol{\lambda} = (\lambda_i : i = 1, \dots, m) \succeq 0$, $\boldsymbol{\nu} = (\nu_1, \dots, \nu_p)$

- Lagrange dual function:

$$\mathcal{D}(\boldsymbol{\lambda}, \boldsymbol{\nu}) = \inf_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\nu})$$

Lower Bound on the Optimal Value

- The dual function $\mathcal{D}(\boldsymbol{\lambda}, \boldsymbol{\nu})$ is a **lower bound** on the optimal value p^* .
- **Theorem.** $\mathcal{D}(\boldsymbol{\lambda}, \boldsymbol{\nu}) \leq p^*, \quad \forall \boldsymbol{\lambda} \succeq 0, \boldsymbol{\nu}$
- **Proof.** Consider a feasible $\tilde{\mathbf{x}}$. Then,

$$\mathcal{L}(\tilde{\mathbf{x}}, \boldsymbol{\lambda}, \boldsymbol{\nu}) = f(\tilde{\mathbf{x}}) + \sum_{i=1}^m \lambda_i g_i(\tilde{\mathbf{x}}) + \sum_{i=1}^p \nu_i h_i(\tilde{\mathbf{x}}) \leq f(\tilde{\mathbf{x}})$$

since $g_i(\tilde{\mathbf{x}}) \leq 0, \lambda_i \geq 0$ and $h_i(\tilde{\mathbf{x}}) = 0$.

Hence, $\mathcal{D}(\boldsymbol{\lambda}, \boldsymbol{\nu}) \leq \mathcal{L}(\tilde{\mathbf{x}}, \boldsymbol{\lambda}, \boldsymbol{\nu}) \leq f(\tilde{\mathbf{x}})$ for all feasible $\tilde{\mathbf{x}}$. Therefore, $\mathcal{D}(\boldsymbol{\lambda}, \boldsymbol{\nu}) \leq p^*$.

- **Question.** What's the best lower bound?

- Dual variables: (λ, ν)

- L7(2)

- What's the relationship between d^* and p^* ?

Weak Duality

$$d^* \leq p^*$$

- Weak duality **always** hold (even if the primal problem is not convex):
- Optimal duality gap: $p^* - d^*$
- Efficient generation of the lower bounds through the dual problem

- (1) Optimization Using Gradient Descent
- (2) Constrained Optimization and Lagrange Multipliers
- (3) **Convex Sets and Functions**
- (4) Convex Optimization

- Convex optimization problem

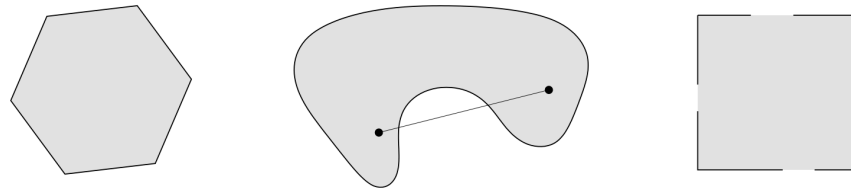
minimize $f(\mathbf{x})$

subject to $\mathbf{x} \in \mathcal{X}$,

where $f(\mathbf{x}) : \mathbb{R}^n \mapsto \mathbb{R}$ is a convex function, and $\mathcal{X}$ is a convex set.

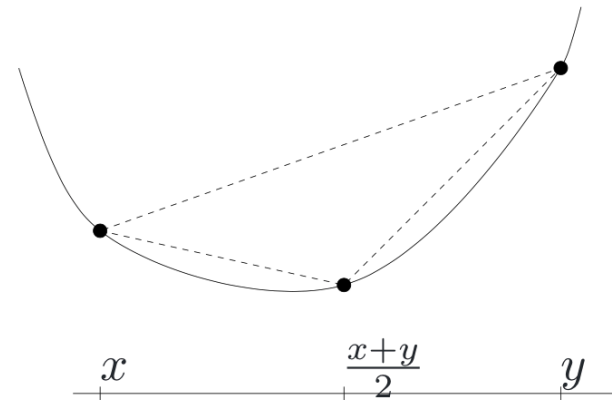
- The watershed between easily solvable problem and intractable ones is not 'linearity', but '**convexity**'
- Let's overview the background of convex functions, convex sets, and their basic properties.

- Set $\mathcal{C}$ is a **convex set** if the line segment between any two points in $\mathcal{C}$ lies in $\mathcal{C}$, i.e., if for any $x_1, x_2 \in \mathcal{C}$ and any $\theta \in [0, 1]$, we have $\theta x_1 + (1 - \theta)x_2 \in \mathcal{C}$
- Examples of convex and non-convex sets



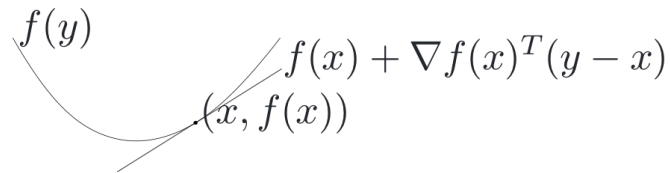
Convex Functions

- $f : \mathbb{R}^n \mapsto \mathbb{R}$ is a **convex function** if $\text{dom } f$ is a convex set and for all $x, y \in \text{dom } f$ and $\theta \in [0, 1]$, we have
$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y)$$
- f is **strictly convex** if the strict inequality in the above holds for all $x \neq y$ and $0 < \theta < 1$.
- f is **concave** if $-f$ is convex
- Affine functions are convex and concave
- **Jensen's inequality.** For a rv X ,
$$f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)].$$



Conditions of Convex Functions (1)

- **First-order condition.** For differentiable functions, f is convex iff
$$f(y) - f(x) \geq \nabla f(x)^T (y - x), \quad \forall x, y \in \text{dom } f, \text{ and } \text{dom } f \text{ is convex}$$



- **Example.** $f(y) = y^2$.
- $f(y) \geq \tilde{f}_x(y)$ where $\tilde{f}_x(y)$ is the first order Taylor expansion of $f(y)$ at x .
- **Local** information (first order Taylor approximation) about a convex function provides **global** information (global underestimator).
- If $\nabla f(x) = 0$, then $f(y) \geq f(x)$, $\forall y$. Thus, x is a global minimizer of f

Conditions of Convex Functions (2)

- **Second-order condition.** For twice differentiable functions, f is convex iff $\nabla^2 f(x) \succeq 0$
for all $x \in \text{dom } f$ (upward slope) and $\text{dom } f$ is convex
- Example: $f(x) = x^2$.
- Meaning: The graph of the function have positive (upward) curvature at x .

Examples of Convex or Concave Functions

- e^{ax} is convex on $\mathbb{R}$, for any $a \in \mathbb{R}$
- x^a is convex on $\mathbb{R}_{++}$ when $a \geq 1$ or $a \leq 0$, and concave for $0 \leq a \leq 1$
- $|x|^p$ is convex on $\mathbb{R}$ for $p \geq 1$
- $\log x$ is concave on $\mathbb{R}_{++}$
- $x \log x$ is strictly convex on $\mathbb{R}_{++}$
- Every norm on $\mathbb{R}^n$ is convex
- $f(x) = \max\{x_1, \dots, x_n\}$ is convex on $\mathbb{R}^n$
- $f(x) = \log \sum_{i=1}^n e^{x_i}$ is convex on $\mathbb{R}^n$
- $f(x) = (\prod_{i=1}^n x_i)^{\frac{1}{n}}$ is concave on $\mathbb{R}_{++}^n$

- $f = \sum_{i=1}^n w_i f_i$ convex if f_i are all convex and $w_i \geq 0$
- $g(x) = f(Ax + b)$ is convex iff $f(x)$ is convex
- $f(x) = \max\{f_1(x), f_2(x)\}$ convex if f_i convex, e.g., sum of r largest components is convex
- $f(x) = h(g(x))$, where $h : \mathbb{R}^k \mapsto \mathbb{R}$ and $g : \mathbb{R}^n \mapsto \mathbb{R}^k$.

If $k = 1$: $f''(x) = h''(g(x))g'(x)^2 + h'(g(x))g''(x)$. So, f is convex if h is convex and nondecreasing and g is convex, or if h is convex and nonincreasing and g is concave ...

- If $f(x, y)$ is convex in x for each $y \in \mathcal{A}$, then

$$g(x) = \sup_{y \in \mathcal{A}} f(x, y)$$

is **convex**. Similarly, if $f(x, y)$ is concave in x for each $y \in \mathcal{A}$, then

$$g(x) = \inf_{y \in \mathcal{A}} f(x, y)$$

is **concave**.

- **Example.** distance to farthest point in a set $\mathcal{C}$: $f(x) = \sup_{y \in \mathcal{C}} \|x - y\|$ is **convex**.
- **Example.** Lagrange dual function

$$\mathcal{D}(\lambda, \nu) = \inf_x \mathcal{L}(x, \lambda, \nu)$$

is **concave**.

- (1) Optimization Using Gradient Descent
- (2) Constrained Optimization and Lagrange Multipliers
- (3) Convex Sets and Functions
- (4) **Convex Optimization**

- A **standard convex optimization** problem with variables $\mathbf{x}$:

$$\begin{aligned} &\text{minimize} && f(\mathbf{x}) \\ &\text{subject to} && g_i(\mathbf{x}) \leq 0, \quad i = 1, 2, \dots, m \\ & && a_i^\top \mathbf{x} = b_i, \quad i = 1, 2, \dots, p \end{aligned}$$

where $f, g_1, \dots, g_m$ are convex functions.

- **Minimize convex** objective function (or maximize concave objective function)
- **Upper bound inequality** constraints on **convex** functions ($\Rightarrow$ Constraint set is convex)
- **Equality** constraints must be **affine** (Only affine functions leads to a convex set for the equality constraints)

- Strong duality (zero optimal duality gap):

$$d^* = p^*$$

- If strong duality holds, solving dual is 'equivalent' to solving primal. But strong duality does **not** always hold
- Convexity and **constraint qualifications** $\implies$ Strong duality
- Another reason why convex optimization is 'easy'

KKT Condition

- Since $\mathbf{x}^*$ minimizes $\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}^*, \boldsymbol{\nu}^*)$ over $\mathbf{x}$,

$$\nabla f(\mathbf{x}^*) + \sum_{i=1}^m \lambda_i^* \nabla g_i(\mathbf{x}^*) + \sum_{i=1}^p \nu_i^* \nabla h_i(\mathbf{x}^*) = 0$$

Karush-Kuhn-Tucker optimality condition

$$g_i(\mathbf{x}^*) \leq 0, \quad h_i(\mathbf{x}^*) = 0, \quad \lambda_i^* \succeq 0$$

$$\lambda_i^* g_i(\mathbf{x}^*) = 0$$

$$\nabla f(\mathbf{x}^*) + \sum_{i=1}^m \lambda_i^* \nabla g_i(\mathbf{x}^*) + \sum_{i=1}^p \nu_i^* \nabla h_i(\mathbf{x}^*) = 0$$

- **Any** optimization with strong duality, KKT condition is necessary for primal-dual optimality
- **Convex** optimization (with Slater's condition), KKT is also **sufficient** for primal-dual optimality.

- Primal problem

$$\begin{array}{ll}\min_{\mathbf{x} \in \mathbb{R}^d} & \mathbf{c}^\top \mathbf{x} \\ \text{subject to} & \mathbf{Ax} \preceq \mathbf{b},\end{array}$$

where $\mathbf{A} \in \mathbb{R}^{m \times d}$ and $\mathbf{b} \in \mathbb{R}^m$.

- Dual problem

$$\begin{array}{ll}\max_{\boldsymbol{\lambda} \in \mathbb{R}^m} & -\mathbf{b}^\top \boldsymbol{\lambda} \\ \text{subject to} & \mathbf{c} + \mathbf{A}^\top \boldsymbol{\lambda} = \mathbf{0}, \boldsymbol{\lambda} \succeq \mathbf{0},\end{array}$$

where $\boldsymbol{\lambda} \in \mathbb{R}^m$.

- The Lagrangian: $\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = (\mathbf{c} + \mathbf{A}^\top \boldsymbol{\lambda})^\top \mathbf{x} - \boldsymbol{\lambda}^\top \mathbf{b}$, whose derivative w.r.t. $\mathbf{x}$ becomes zero, when $\mathbf{c} + \mathbf{A}^\top \boldsymbol{\lambda} = \mathbf{0}$.
- The dual function: $\mathcal{D}(\boldsymbol{\lambda}) = -\boldsymbol{\lambda}^\top \mathbf{b}$

- Primal problem

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^d} \quad & \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{c}^T \mathbf{x} \\ \text{subject to} \quad & \mathbf{A} \mathbf{x} \preceq \mathbf{b}, \end{aligned}$$

where $\mathbf{A} \in \mathbb{R}^{m \times d}$, $\mathbf{b} \in \mathbb{R}^m$, $\mathbf{c} \in \mathbb{R}^d$, the square matrix $\mathbf{Q}$ is symmetric, positive definite.

- Dual problem

$$\begin{aligned} \max_{\boldsymbol{\lambda} \in \mathbb{R}^m} \quad & \left(-\frac{1}{2} (\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda})^T \mathbf{A} \mathbf{Q}^{-1} (\mathbf{c} + \mathbf{A}^T \boldsymbol{\lambda}) - \boldsymbol{\lambda}^T \mathbf{b} \right) \\ \text{subject to} \quad & \boldsymbol{\lambda} \succeq 0, \end{aligned}$$

where $\boldsymbol{\lambda} \in \mathbb{R}^m$.

- (1) Optimization Using Gradient Descent
- (2) Constrained Optimization and Lagrange Multipliers
- (3) Convex Sets and Functions
- (4) Convex Optimization

Questions?

Lecture 3: Principal Component Analysis

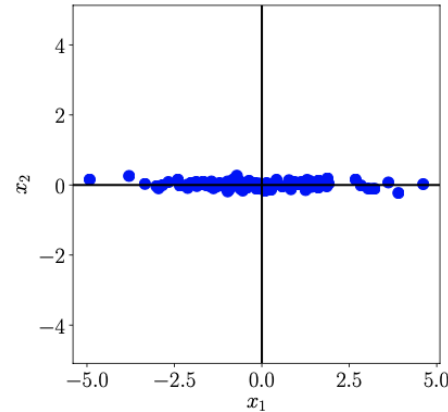
Jinwoo Shin

Kim Jaechul Graduate School of AI, KAIST

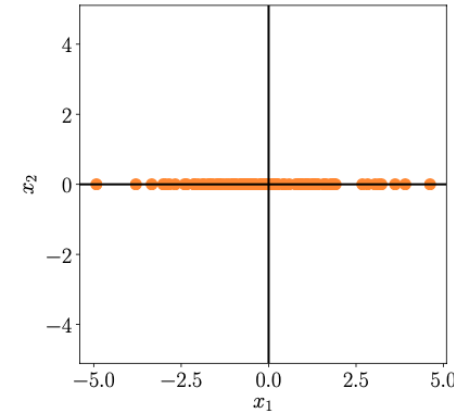
- (1) Problem Setting
- (2) Maximum Variance Perspective
- (3) Eigenvector Computation and Low-Rank Approximations
- (4) PCA in High Dimensions

- (1) Problem Setting
- (2) Maximum Variance Perspective
- (3) Eigenvector Computation and Low-Rank Approximations
- (4) PCA in High Dimensions

Dimensionality Reduction



(a) Dataset with x_1 and x_2 coordinates.



(b) Compressed dataset where only the x_1 coordinate is relevant.

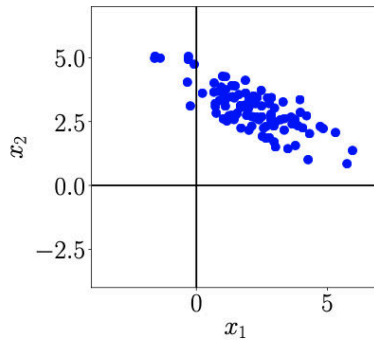
- High-dimensional data
 - hard to analyze and visualize
 - Often, overcomplete and many dimensionas are redundant
- Compact data representation is always preferred just like compression.
- PCA (Principal Component Analysis) is a representative method.

Example: Housing Data

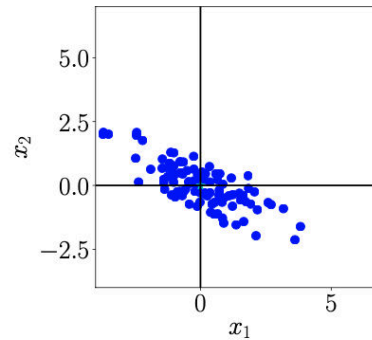
- 5 dimensions
 1. Size
 2. Number of rooms
 3. Number of bathrooms
 4. Schools around
 5. Crime rate
- 2 dimensions
 - Size feature
 - Location feature

- S1. Centering.** Centering the data by subtracting mean
- S2. Standardization.** Divide the data points by the standard deviation for every dimension (original feature) $d = 1, \dots, D$
- S3. Eigenvalue/vector.** Compute the M -largest eigenvalues and the eigenvectors of the data covariance matrix (M is the dimension that needs to be reduced)
- S4. Projection.** Project all data points onto the space defined by the eigenvectors (i.e., principal subspace).
- S5.** Undo standardization and centering.

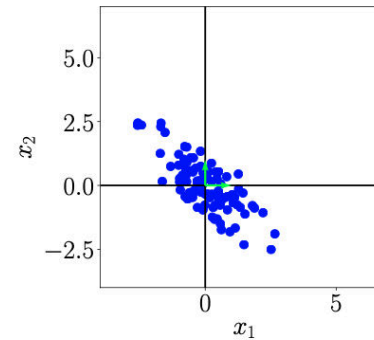
PCA Illustration



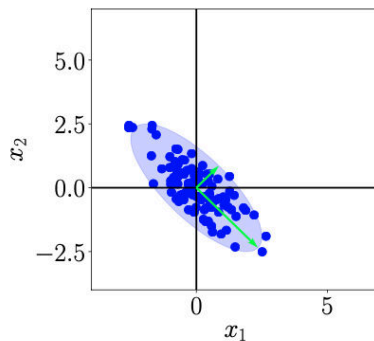
(a) Original dataset.



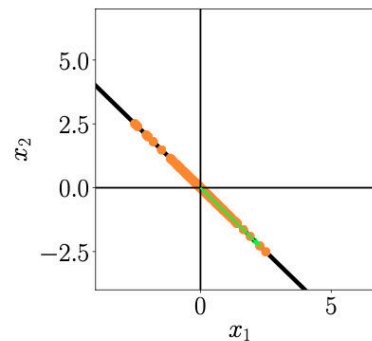
(b) Step 1: Centering by subtracting the mean from each data point.



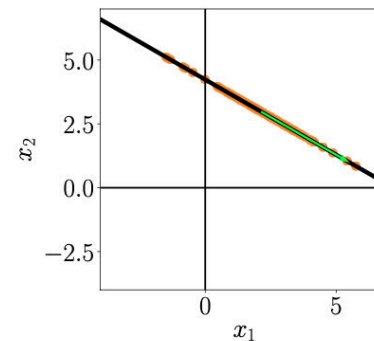
(c) Step 2: Dividing by the standard deviation to make the data unit free. Data has variance 1 along each axis.



(d) Step 3: Compute eigenvalues and eigenvectors (arrows) of the data covariance matrix (ellipse).



(e) Step 4: Project data onto the principal subspace.



(f) Undo the standardization and move projected data back into the original data space from (a).

Data Matrix and Data Covariance Matrix

- N : number of samples, D : number of measurements (or original features)
- iid dataset $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ whose mean is 0 (well-centered), where each $\mathbf{x}_i \in \mathbb{R}^D$, and its corresponding data matrix

$$\mathbf{X} = (\mathbf{x}_1 \ \cdots \ \mathbf{x}_N) = \begin{pmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,N} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,N} \\ \vdots & \vdots & \ddots & \vdots \\ x_{D,1} & x_{D,2} & \cdots & x_{D,N} \end{pmatrix} \in \mathbb{R}^{D \times N}$$

- (data) covariance matrix

$$\mathbf{S} = \frac{1}{N} \mathbf{X} \mathbf{X}^T = \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n \mathbf{x}_n^T \in \mathbb{R}^{D \times D}$$

Code: Low Dimensional Representation

- Low-dimensional compressed representation, also called **code**:

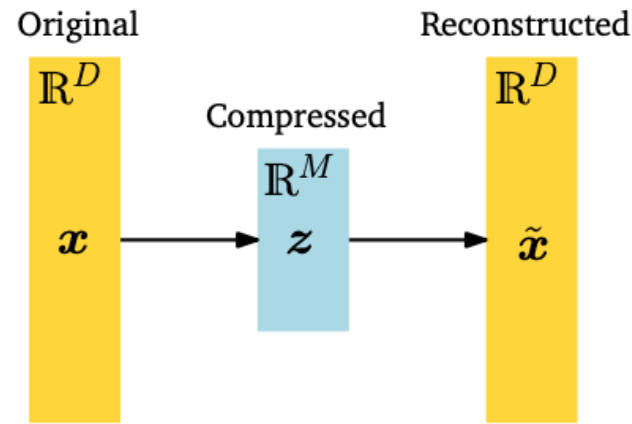
$$\mathbf{z}_n = \mathbf{B}^\top \mathbf{x}_n \in \mathbb{R}^M,$$

where the projection¹ matrix is $\mathbf{B} := (\mathbf{b}_1, \dots, \mathbf{b}_M) \in \mathbb{R}^{D \times M}$,

- Assume that the columns of $\mathbf{B}$ are orthonormal, i.e., $\mathbf{b}_i^\top \mathbf{b}_j = 0$ if $i \neq j$, and $\mathbf{b}_i^\top \mathbf{b}_i = 1$ if $i = j$.
- Seek an M -dimensional subspace $U \subset \mathbb{R}^D$, $\dim(U) = M < D$ onto which we project data
- $\tilde{\mathbf{x}}_n \in \mathbb{R}^D$: projected data, $\mathbf{z}_n$: their coordinates w.r.t. the basis vectors of $\mathbf{B}$.

¹In **L3(8)**, the coordinate in the projected space becomes $\lambda = (\mathbf{B}^\top \mathbf{B})^{-1} \mathbf{B}^\top \mathbf{x}$, which is simply $\mathbf{B}^\top \mathbf{x}$ for orthonormal bases $\mathbf{B}$.

PCA: Encoder and Decoder Viewpoint

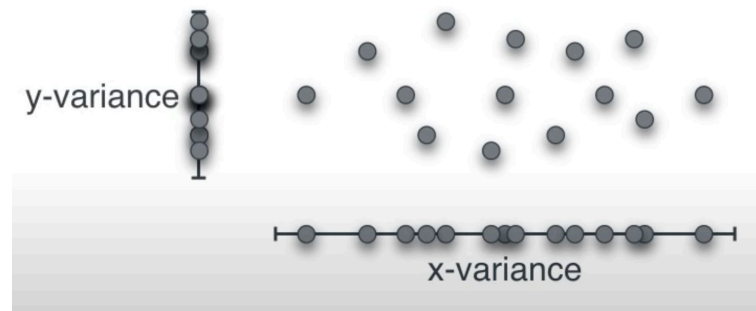


- Find a suitable matrix B such that $z = B^T x$ and $\tilde{x} = Bz$
- B^T : encoder, B : decoder
- **Example.** MNIST dataset
 - handwritten digits, $N = 60,000$ data samples, $D = 28 \times 28 = 784$ pixels



- (1) Problem Setting
- (2) Maximum Variance Perspective
- (3) Eigenvector Computation and Low-Rank Approximations
- (4) PCA in High Dimensions

- Information content in the data
 - space filling
 - information in the data by looking at how much data is spread out
- PCA
 - a dimensionality reduction algorithm that maximizes the variance in the low-dimensional data representation.



source: Youtube channel by Luis Serrano

Matrix Again: B , z_n , and x_n

- $B = (\mathbf{b}_1 \ \mathbf{b}_2 \ \dots \ \mathbf{b}_M)$, where $\mathbf{b}_i \in \mathbb{R}^D$ and $B \in \mathbb{R}^{D \times M}$
- $B^T = \begin{pmatrix} \mathbf{b}_1^T \\ \vdots \\ \mathbf{b}_M^T \end{pmatrix} \in \mathbb{R}^{M \times D}$, $\mathbf{b}_i^T \in \mathbb{R}^{1 \times D}$, $\mathbf{x}_i \in \mathbb{R}^{D \times 1}$
- $\mathbf{z}_n = \begin{pmatrix} z_{1n} \\ \vdots \\ z_{Mn} \end{pmatrix} = B^T \mathbf{x}_n = \begin{pmatrix} \mathbf{b}_1^T \\ \vdots \\ \mathbf{b}_M^T \end{pmatrix} \mathbf{x}_n = \begin{pmatrix} \mathbf{b}_1^T \mathbf{x}_n \\ \vdots \\ \mathbf{b}_M^T \mathbf{x}_n \end{pmatrix}$
- z_{in} : new coordinate (for $\mathbf{x}_n$) in the projected space by the basis $\mathbf{b}_i$

What We Will Do Is ...

- **Goal:** Find the orthonormal bases $\mathbf{B} = (\mathbf{b}_1 \ \mathbf{b}_2 \ \dots \ \mathbf{b}_M)$ that maximizes the variance.
- **Result:** For the M -largest eigenvalues $\lambda_1, \dots, \lambda_M$ of the data covariance matrix $\mathbf{S}$, their corresponding M eigenvectors become $\mathbf{b}_1, \dots, \mathbf{b}_M$
- **Question.** Why data covariance matrix? Why eigenvectors ordered by their eigenvalues?
- **Strategy: Induction**
 - Step 1.** We seek a single vector $\mathbf{b}_1$ that maximizes the variance of the projected data, assuming that we project the data onto an 1D line. We show that $\mathbf{b}_1$ is the **eigenvector of the largest eigenvalue**.
 - Step k.** Suppose that we found $\mathbf{b}_1, \dots, \mathbf{b}_{k-1}$ for the variance maximization. Then, we seek $\mathbf{b}_k$ that maximizes the variance of the projected data onto k -D plain with the constraint that $\mathbf{b}_k$ is orthogonal to $\mathbf{b}_1, \dots, \mathbf{b}_{k-1}$. We prove that $\mathbf{b}_k$ is the **eigenvector of the k -th largest eigenvalue**.

Step 1: Finding $\mathbf{b}_1$ (1)

- Variance (over N sample data) of the first coordinate z_1 of $\mathbf{z} \in \mathbb{R}^M$, so that

$$V_1 := \text{var}[z_1] = \frac{1}{N} \sum_{n=1}^N z_{1n}^2, \quad z_{1n} = \mathbf{b}_1^\top \mathbf{x}_n$$

where z_{1n} (z_{in}) is the first (i -th) coordinate of the low-dimensional representation $\mathbf{z}_n$ of $\mathbf{x}_n$

$$V_1 = \frac{1}{N} \sum_{n=1}^N (\mathbf{b}_1^\top \mathbf{x}_n)^2 = \frac{1}{N} \sum_{n=1}^N \mathbf{b}_1^\top \mathbf{x}_n \mathbf{x}_n^\top \mathbf{b}_1 = \mathbf{b}_1^\top \left(\frac{1}{N} \sum_{n=1}^N \mathbf{x}_n \mathbf{x}_n^\top \right) \mathbf{b}_1 = \mathbf{b}_1^\top \mathbf{S} \mathbf{b}_1$$

- Find $\mathbf{b}_1$ that maximizes V_1 .

$$\max_{\mathbf{b}_1} \mathbf{b}_1^\top \mathbf{S} \mathbf{b}_1, \quad \text{subject to} \quad \|\mathbf{b}_1\|^2 = 1$$

Step 1: Finding $\mathbf{b}_1$ (2)

- Optimization problem

$$\max_{\mathbf{b}_1} \mathbf{b}_1^T \mathbf{S} \mathbf{b}_1, \quad \text{subject to} \quad \|\mathbf{b}_1\|^2 = 1$$

- Using the Lagrange multiplier method, we get:

L7(2), L7(4)

$$\mathbf{S} \mathbf{b}_1 = \lambda_1 \mathbf{b}_1, \quad \mathbf{b}_1^T \mathbf{b}_1 = 1 \implies \lambda_1: \text{eigenvalue}, \mathbf{b}_1: \text{eigenvector of } \mathbf{S}$$

- Then, $V_1 = \mathbf{b}_1^T \mathbf{S} \mathbf{b}_1 = \lambda_1 \mathbf{b}_1^T \mathbf{b}_1 = \lambda_1$ (the variance V_1 is the eigenvalue of $\mathbf{S}$)
- To maximize the variance, we take the largest eigenvalue, and the corresponding eigenvector is called the (first) principal component.

Step k : Finding $\mathbf{b}_k$ (1)

- Finding k -th principal component: Solving the following optimization problem

$$\max_{\mathbf{b}} \mathbf{b}^T \mathbf{S} \mathbf{b}, \quad \text{subject to} \quad \mathbf{b}^T \mathbf{b} = 1 \text{ and } \mathbf{b}^T \mathbf{b}_i = 0, \quad i = 1, \dots, k-1$$

- Claim.** The solution of the above is the eigenvector of $\mathbf{S}$ corresponding to its k -th largest eigenvalue.
- Proof.** By induction hypothesis, $\mathbf{b}_1, \dots, \mathbf{b}_k$ are the orthonormal eigenvectors of $\mathbf{S}$. Denote the i -th largest eigenvalue of $\mathbf{S}$ by λ_i , where note that $\mathbf{S} \mathbf{b}_i = \lambda_i \mathbf{b}_i$. The lagrangian of the objective function is:

$$\mathcal{L}(\mathbf{b}) = \mathbf{b}^T \mathbf{S} \mathbf{b} - \lambda(\mathbf{b}^T \mathbf{b} - 1) + \sum_{i=1}^k \eta_i \mathbf{b}^T \mathbf{b}_i$$

Step k : Finding $\mathbf{b}_k$ (2)

- Letting the solution be denoted by $\mathbf{b}_{k+1}$, the first-order necessary condition for optimality is:

$$\nabla \mathcal{L}(\mathbf{b}_{k+1}) = 2\mathbf{S}\mathbf{b}_{k+1} - 2\lambda\mathbf{b}_{k+1} + \sum_{i=1}^k \eta_i \mathbf{b}_i = 0 \quad (*)$$

- Now, for any $j \in \{1, \dots, k\}$,

$$\begin{aligned} 0 &= \mathbf{b}_j^\top \nabla \mathcal{L}(\mathbf{b}_{k+1}) = 2\mathbf{b}_j^\top \mathbf{S}\mathbf{b}_{k+1} - 2\lambda \mathbf{b}_j^\top \mathbf{b}_{k+1} + \sum_{i=1}^k \eta_i \mathbf{b}_j^\top \mathbf{b}_i = 2(\mathbf{S}\mathbf{b}_j)^\top \mathbf{b}_{k+1} + \eta_j \\ &= 2(\lambda \mathbf{b}_j)^\top \mathbf{b}_{k+1} + \eta_j = 2\lambda \mathbf{b}_j^\top \mathbf{b}_{k+1} + \eta_j = \eta_j \end{aligned}$$

- From $\eta_j = 0$ and $(*)$, $\mathbf{S}\mathbf{b}_{k+1} = \lambda\mathbf{b}_{k+1}$. $\implies \lambda$ is an eigenvalue and its corresponding eigenvector is $\mathbf{b}_{k+1}$.
- Note that the objective function is λ , because $\mathbf{b}^\top \mathbf{S}\mathbf{b} = \lambda \mathbf{b}^\top \mathbf{b}$.

Step k : Finding $\mathbf{b}_k$ (3)

- **Question.** How can we choose the largest λ with the constraint that $\mathbf{b}_{k+1} \perp (\mathbf{b}_1, \dots, \mathbf{b}_k)$?
- Clearly, if $\mathbf{b}_{k+1}$ is equal to any of these eigenvectors (up to sign), the constraint will be violated, so, to maximize λ , $\mathbf{b}_{k+1}$ should be a unit eigenvector of $\mathbf{S}$ corresponding to $(k+1)$ -th largest eigenvalue.
- By spectral theorem, we can choose this vector in such a way that it is orthogonal to $\mathbf{b}_1, \dots, \mathbf{b}_k$.

L4(4)

- (1) Problem Setting
- (2) Maximum Variance Perspective
- (3) Eigenvector Computation and Low-Rank Approximations
- (4) PCA in High Dimensions

- Approach 1: EVD

L4(4)

- Perform an eigendecomposition and compute the eigenvalues and eigenvectors of the symmetric matrix $\mathbf{S}$ directly.

- Approach 2: SVD

L4(5)

- SVD of the data matrix $\mathbf{X}$: $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ ($[D \times N] = [D \times D] \cdot [D \times N] \cdot [N \times N]$)
- $\mathbf{U}$ and $\mathbf{V}^T$: orthogonal matrices, $\mathbf{\Sigma}$: only nonzero entries are the singular values $\sigma_{ii} \geq 0$.

$$\mathbf{S} = \frac{1}{N} \mathbf{X}\mathbf{X}^T = \frac{1}{N} \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \mathbf{V}\mathbf{\Sigma}^T \mathbf{U}^T \stackrel{(\mathbf{V}^T = \mathbf{V}^{-1})}{=} \frac{1}{N} \mathbf{U}\mathbf{\Sigma}\mathbf{\Sigma}^T \mathbf{U}^T$$

- The columns of $\mathbf{U}$ are the eigenvectors of $\mathbf{X}\mathbf{X}^T$ (thus $\mathbf{S}$)
- The eigenvalues λ_d of $\mathbf{S}$ are related to the singular values of $\mathbf{X}$: $\lambda_d = \frac{\sigma_d^2}{N}$

PCA as Low-Rank Matrix Approximations

- In SVD, $\mathbf{U}$ corresponds to the projection matrix $\mathbf{B}$, so that we maximize the variance of the projected data or minimize the average squared reconstruction error.
- Consider the best rank- M approximation

$$\tilde{\mathbf{X}}_M := \arg \min_{\text{rk}(\mathbf{A})=M} \|\mathbf{X} - \mathbf{A}\|_2$$

- From Eckart-Young Theorem, by truncating the SVD at the top- M singular value, we obtain the reconstructed data matrix $\tilde{\mathbf{X}}_M$ as: L4(5), L4(6)

$$\tilde{\mathbf{X}}_M = \overbrace{\mathbf{U}_M}^{D \times M} \overbrace{\Sigma_M}^{M \times M} \overbrace{\mathbf{V}_M^T}^{M \times N} \iff \tilde{\mathbf{X}}_M = \sum_{i=1}^M \sigma_i \mathbf{u}_i \mathbf{v}_i^T,$$

where σ_i is the i -th singular value.

- (1) Problem Setting
- (2) Maximum Variance Perspective
- (3) Eigenvector Computation and Low-Rank Approximations
- (4) **PCA in High Dimensions**

- In some practical cases, $\mathbf{S} = \frac{1}{N}\mathbf{X}\mathbf{X}^\top \in \mathbb{R}^{D \times D}$, where D is pretty high.
 - **Example.** 100×100 pixel image: $D = 10,000$.
- What if $N \ll D$?
 - With no duplicate data, $\text{rk}(\mathbf{S}) = N$, and $D - N + 1$ eigenvalues are 0! $\implies$ no need to maintain $D \times D$ data covariance matrix.
- In PCA, $\mathbf{S}\mathbf{b}_m = \lambda_m \mathbf{b}_m$, $m = 1, \dots, M$.

$$\mathbf{S}\mathbf{b}_m = \frac{1}{N}\mathbf{X}\mathbf{X}^\top \mathbf{b}_m = \lambda_m \mathbf{b}_m \implies \frac{1}{N} \underbrace{\mathbf{X}^\top \mathbf{X}}_{N \times N} \underbrace{\mathbf{X}^\top \mathbf{b}_m}_{:= \mathbf{c}_m} = \lambda_m \mathbf{X}^\top \mathbf{b}_m \iff \frac{1}{N} \mathbf{X}^\top \mathbf{X} \mathbf{c}_m = \lambda_m \mathbf{c}_m$$

- λ_m is an eigenvalue of $\frac{1}{N}\mathbf{X}^\top \mathbf{X}$ with its associated eigenvector $\mathbf{c}_m = \mathbf{X}^\top \mathbf{b}_m$
- $\frac{1}{N}\mathbf{X}^\top \mathbf{X} \in \mathbb{R}^{N \times N}$, so much easier to compute the eigenstuff
- To recover the eigenvector of $\mathbf{S}$, by left-multiplying $\mathbf{X}$, we get $\frac{1}{N}\mathbf{X}\mathbf{X}^\top \mathbf{X}\mathbf{c}_m = \lambda_m \mathbf{X}\mathbf{c}_m$

Questions?